CE4052/CZ4052/SC4052 Cloud Computing

Semester 2 Examination 2023–2024 — Model Answers

Question 1

(a) Para-virtualization and Hardware-assisted vs Full Virtualization (10 marks)

Full Virtualization (baseline) The hypervisor traps and emulates every privileged instruction issued by the unmodified guest OS using binary translation. This guarantees compatibility with any standard OS but incurs performance overhead from the translation layer.

Para-Virtualization — Advantages over Full Virtualization

Advantage	Explanation
Eliminates binary translation	Guest OS is modified to issue **hypercalls** directly to the hypervisor instead of privileged instructions. No runtime translation overhead.
Better I/O performance	PV drivers use shared-memory interfaces, avoiding emulated device overhead.
Lower hypervisor complexity	The hypervisor does not need to maintain a full hardware emulation layer.

Disadvantages of Para-Virtualization over Full Virtualization | Disadvantage | Explanation | |---|---| | Requires guest OS modification | The guest kernel must be ported to call the hypervisor API (e.g., Xen hypercalls). Proprietary OSes (e.g., unmodified Windows) cannot be used. | | Higher porting/maintenance effort | Every supported OS version needs its own set of PV drivers. | | Less hardware portability | PV guests are tied to the specific hypervisor ABI. |

Hardware-Assisted Virtualization — Advantages over Full Virtualization

Advantage	Explanation
No binary translation required	CPU extensions (Intel VT-x / AMD-V) provide a new privilege ring (VMX root mode) where the hypervisor runs. Sensitive instructions trap automatically in hardware.
Runs **unmodified** guest OS	No guest changes needed — maximum OS compatibility, like full virtualization, but faster.
Best practical performance	Hardware handles context switching between guest and hypervisor, dramatically reducing overhead.
Hardware-enforced isolation	Memory and I/O protection (EPT/SLAT, IOMMU) is managed by the CPU, improving security.

Disadvantages of Hardware-Assisted Virtualization over Full Virtualization | Disadvantage | Explanation | |---|---| | Requires compatible hardware | Older CPUs without VT-x/AMD-V cannot use it; full virtualization works on any x86 CPU. | | Early implementations had VM-exit overhead | Some workloads with frequent privileged instruction calls can generate many VM exits, incurring context-switch cost (though this has improved greatly in modern CPUs). |

Summary: Para-virtualization trades OS compatibility for performance through software cooperation. Hardware-assisted virtualization achieves near-native performance without any OS modification, and is now the dominant approach in production data centres (e.g., AWS, Google Cloud, Azure all rely on HW-assisted VMs).

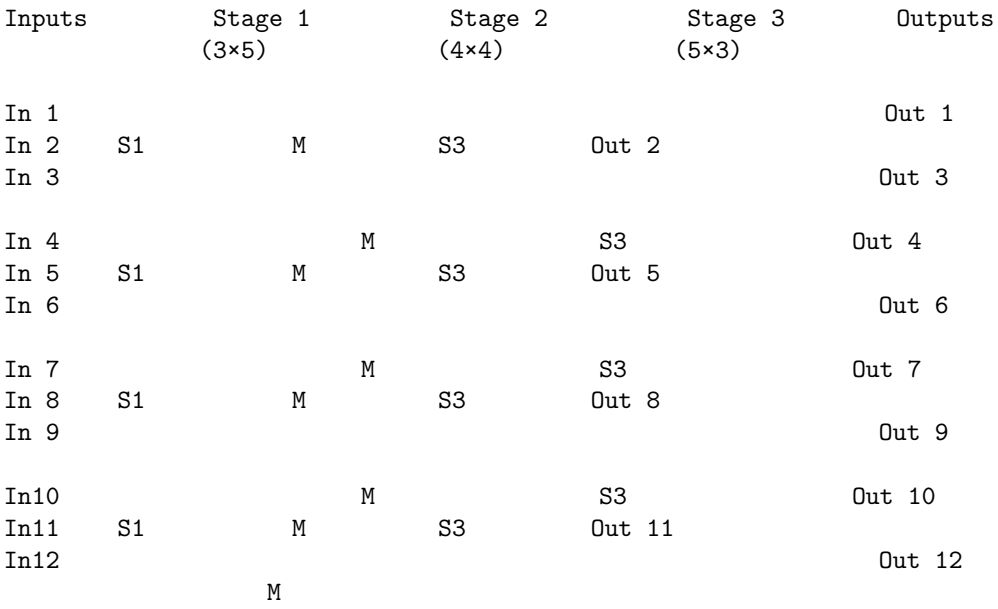
(b) 3-stage Close Network: N=12, n=3, k=5, m=4 (15 marks)

Parameters

- **Stage 1 (ingress):** $r = N/n = 12/3 = 4$ switches, each of size $n \times k = 3 \times 5$
- **Stage 2 (middle):** $k = 5$ switches, each of size $m \times m = 4 \times 4$
- **Stage 3 (egress):** $r = 4$ switches, each of size $k \times n = 5 \times 3$

  Note: For a strictly non-blocking Close network, the condition $m \geq 2n-1 = 5$ must hold. Here $m = 4 < 5$, so this is a **rearrangeably non-blocking** Close network.

Topology Diagram



Each of the 4 Stage-1 switches connects to **all 5** middle switches (one link each). Each of the 5 middle switches connects to **all 4** Stage-3 switches.

Switch Count

Stage	Count	Size
Stage 1 (ingress)	4	3×5
Stage 2 (middle)	5	4×4
Stage 3 (egress)	4	5×3
Total	**13**	

Advantage of Close Networks in Data Centres

1. **Multiple equal-cost paths:** Any source can reach any destination via any of the $k = 5$ middle switches. This enables ECMP load balancing and higher aggregate bandwidth.
2. **Scalability with commodity switches:** The Close topology scales by adding more switches rather than replacing core switches with higher-radix ones, making it cost-effective.
3. **Fault tolerance:** Path redundancy means failure of a single middle switch only reduces available bandwidth, it does not disconnect any source-destination pair (rearrangeably non-blocking).
4. **Higher bisection bandwidth:** Compared to a simple tree, the Close provides far more bandwidth between arbitrary halves of the network.

Question 2

(a) RAID5 Recovery + Comparison (10 marks)

Setup

- $n = 6$ disks; each disk fails independently with $p = 0.003$
- RAID5 uses one distributed parity block per stripe across all 6 disks (5 data + 1 parity per stripe, parity position rotates)

Recovery from One Failed Disk in RAID5 For each stripe, the parity block P is defined as:

$$P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5$$

If disk i (containing D_i) fails, reconstruct it as:

$$D_i = P \oplus D_1 \oplus \cdots \oplus D_{i-1} \oplus D_{i+1} \oplus \cdots \oplus D_5$$

i.e., XOR all surviving blocks including parity. No data is permanently lost as long as at most one disk fails.

Probability of Data Loss Comparison **RAID 0 (striping only):** Data lost if **any** disk fails.

$$P_{\text{loss,RAID0}} = 1 - (1-p)^6 \approx 6p \quad \text{(for small } p\text{)}$$

RAID 5 (n=6, tolerates 1 failure): Data lost if **2 or more** disks fail.

$$P_{\text{loss,RAID5}} = 1 - (1-p)^6 - 6p(1-p)^5 \approx \binom{6}{2} p^2 = 15p^2$$

RAID 1 with 3× replication: With 6 disks and 3-way replication, we store 2 unique data units, each replicated on 3 disks. Data is lost only when all 3 copies of any unit fail.

$$P_{\text{loss,RAID1(3×)}} \approx 2p^3$$

Scheme	P(data loss) approx.	Unique capacity
RAID 0	$6p = 0.018$	$6S$ (full)
RAID 5	$15p^2 = 1.35 \times 10^{-4}$	$5S$
RAID 1 (3×)	$2p^3 = 5.4 \times 10^{-8}$	$2S$

RAID 1 (3×) provides the best reliability but wastes 2/3 of raw capacity. RAID 5 balances reliability and capacity. RAID 0 offers no fault tolerance.

(b) More Powerful Erasure Coding vs RAID5 vs RAID0 (15 marks)

New Scheme: Tolerates $t > 1$ Failures (e.g., RAID6 with $t = 2$) With 6 disks and $t = 2$ (protecting against up to 2 failures, using 2 parity blocks):

- Usable capacity: $(6-2)S = 4S$ (vs $5S$ for RAID5)
- Data lost only if **3 or more** disks fail:

$$P_{\text{loss,RAID6}} \approx \binom{6}{3} p^3 = 20p^3$$

Three-way Comparison

Scheme	Parity blocks	Usable capacity	P(loss) approx. for $p = 0.003$
RAID 0	0	$6S$	$6p \approx 1.8 \times 10^{-2}$
RAID 5	1	$5S$	$15p^2 \approx 1.35 \times 10^{-4}$
RAID 6	2	$4S$	$20p^3 \approx 5.4 \times 10^{-7}$

Tradeoff Between Redundancy and Unique Information as p Varies Yes, there is a clear tradeoff:

- **Adding redundancy** (more parity blocks, higher $r = n - k$) exponentially reduces the probability of data loss: $P_{\text{loss}} \propto p^{r+1}$.
- **But each additional parity block reduces** the fraction of disks storing unique data: storage efficiency $= k/n = (n - r)/n$.

For **small** p (e.g., $p = 0.003$): RAID5 already achieves $P_{\text{loss}} \approx 10^{-4}$, which is excellent. Adding a second parity block (RAID6) wastes capacity for marginal reliability gain. RAID5 is preferred.

For **large** p (e.g., $p = 0.1$): $P_{\text{loss,RAID5}} \approx 15\%$, which is unacceptable. RAID6 brings this to $20 \times 10^{-3} = 2\%$. Additional redundancy becomes essential.

Conclusion: As p increases, the optimal choice shifts toward higher redundancy despite the capacity cost. The tradeoff is: **more redundancy ⇒ lower loss probability, but less unique data per disk**.

Question 3

(a) SEDF Schedulability (10 marks)

VMs and their utilizations:

VM	Tuple (s_i, p_i, x_i)	$u_i = s_i/p_i$
VM1	$(1, 8, 1)$	$1/8 = 0.125$
VM2	$(1, 5, 1)$	$1/5 = 0.200$
VM3	$(2, 6, 1)$	$2/6 = 0.333$
VM4	$(3, 15, 1)$	$3/15 = 0.200$
VM5	$(4, 30, 1)$	$4/30 = 0.133$
Total		$U = 0.991 \leq 1$

Yes, they are schedulable since $U = 0.991 \leq 1$.

Hyperperiod $= \text{lcm}(8, 5, 6, 15, 30) = 120$ time units.

CPU quanta per hyperperiod (= total execution time per hyperperiod):

VM	Units/period	Periods in 120	Total CPU units
VM1	1	15	**15**
VM2	1	24	**24**
VM3	2	20	**40**
VM4	3	8	**24**
VM5	4	4	**16**

- **Most CPU:** VM3 (40 units)
- **Least CPU:** VM1 (15 units)

Tie-breaking rule: When two VMs have the same deadline, run the one with the smaller index (VM1 < VM2 < … < VM5).

Sample schedule (first 15 time slots):

t	Earliest deadline	VM run	Notes
0	VM2 ($d = 5$)	VM2	$r_2 \to 0$
1	VM3 ($d = 6$)	VM3	$r_3 \to 1$
2	VM3 ($d = 6$)	VM3	VM3 period 1 done
3	VM1 ($d = 8$)	VM1	$r_1 \to 0$, VM1 done
4	VM4 ($d = 15$)	VM4	$r_4 \to 2$
5	VM2 ($d = 10$, period 2)	VM2	$r_2 \to 0$
6	VM3 ($d = 12$, period 2)	VM3	$r_3 \to 1$
7	VM3 ($d = 12$)	VM3	VM3 period 2 done
8	VM1 ($d = 16$, period 2)	VM4	VM4 $d = 15 <$ VM1 $d = 16$
9	VM4 ($d = 15$)	VM4	VM4 period 1 done
10	VM2 ($d = 15$, period 3)	VM2	$r_2 \to 0$
11	VM1 ($d = 16$)	VM1	VM1 period 2 done
12	VM3 ($d = 18$, period 3)	VM3	$r_3 \to 1$
13	VM3 ($d = 18$)	VM3	VM3 period 3 done
14	VM4 ($d = 30$, period 2)	VM5	VM5 $d = 30$; VM4 $d = 30$ — tie, VM4 runs

The schedule repeats with period 120. All deadlines are met since $U \leq 1$.

(b) Credit Scheduler with VM3 Arrival (10 marks)

Setup: 1 credit = 10ms; 15 credits allocated every 150ms.

Two-VM Case (VM1 weight=256, VM2 weight=512) Total weight $= 256 + 512 = 768$.

$$C_{\text{VM1}} = 15 \times \frac{256}{768} = 5 \text{ credits} = 50\text{ms}$$

$$C_{\text{VM2}} = 15 \times \frac{512}{768} = 10 \text{ credits} = 100\text{ms}$$

Schedule in each 150ms accounting window:

Interval	VM running	Credits consumed	State after
$[0, 50)$ms	VM1	5 credits	VM1 $\to$ *over*
$[50, 150)$ms	VM2	10 credits	VM2 $\to$ *over*
$[150, ...)$	Credits replenished; repeat		

The credit scheduler always runs an *under* VM before an *over* VM. Since VM2 has twice the weight of VM1, VM2 gets twice the CPU share.

When VM3 Arrives (assume weight = 256) New total weight = 256 + 512 + 256 = 1024.

$$C_{\text{VM1}} = 15 \times \frac{256}{1024} = 3.75 \text{ credits} \approx 37.5\text{ms}$$

$$C_{\text{VM2}} = 15 \times \frac{512}{1024} = 7.5 \text{ credits} = 75\text{ms}$$

$$C_{\text{VM3}} = 15 \times \frac{256}{1024} = 3.75 \text{ credits} \approx 37.5\text{ms}$$

Effect on schedule: The Xen credit scheduler handles the arrival dynamically:

1. VM3 starts with full credits (marked *under*), so it is immediately eligible to preempt *over* VMs.
2. In the next accounting window (150ms), credits are recomputed with the new weight sum, automatically redistributing CPU proportionally.
3. VM1 and VM3 each receive ~37.5ms; VM2 receives 75ms per 150ms window — a 1:2:1 ratio.

The global load balancer may also migrate VM3 to a different physical CPU if one is less loaded, exploiting multi-core resources.

Question 4

(a) Diffie-Hellman Key Exchange: p=23, g=5, a=6, b=15 (10 marks)

Step 1 — Alice computes A:

$$A = g^a \bmod p = 5^6 \bmod 23$$

$$5^1 = 5, \quad 5^2 = 25 \equiv 2, \quad 5^3 = 10, \quad 5^6 = (5^3)^2 = 100 \equiv 100 - 4 \times 23 = \mathbf{8}$$

Alice sends $A = 8$ to Bob.

Step 2 — Bob computes B:

$$B = g^b \bmod p = 5^{15} \bmod 23$$

Building up from $5^6 = 8$:

$$5^7 = 40 \equiv 17, \quad 5^8 = 85 \equiv 16, \quad 5^9 = 80 \equiv 11$$
$$5^{10} = 55 \equiv 9, \quad 5^{11} = 45 \equiv 22, \quad 5^{12} = 110 \equiv 18$$
$$5^{13} = 90 \equiv 21, \quad 5^{14} = 105 \equiv 13, \quad 5^{15} = 65 \equiv \mathbf{19}$$

Bob sends $B = 19$ to Alice.

Step 3 — Shared secret:

Alice computes: $s = B^a \bmod p = 19^6 \bmod 23$

$$19^2 = 361 \equiv 16, \quad 19^3 = 19 \times 16 = 304 \equiv 5, \quad 19^6 = (19^3)^2 = 25 \equiv \mathbf{2}$$

Bob computes: $s = A^b \bmod p = 8^{15} \bmod 23$

$$8^2 \equiv 18, \quad 8^4 \equiv 324 \equiv 4, \quad 8^8 \equiv 16$$

$$8^{15} = 8^8 \times 8^4 \times 8^2 \times 8^1 = 16 \times 4 \times 18 \times 8 \bmod 23$$
$$= 64 \times 144 \bmod 23 \equiv 18 \times 8 \equiv 144 \equiv \mathbf{2}$$

Shared secret $s = 2$.

(b) Eve's Cryptanalysis Problem (10 marks)

Eve observes: $p = 23$, $g = 5$, $A = 8$, $B = 19$.

To reconstruct the shared secret, Eve must solve at least one of:

The Discrete Logarithm Problem (DLP):

$$\text{Find } a \text{ such that } g^a \equiv A \pmod{p} \quad \Rightarrow \quad 5^a \equiv 8 \pmod{23}$$

or equivalently find b such that $5^b \equiv 19 \pmod{23}$.

Once a (or b) is known, Eve computes $s = B^a \bmod p$ (or $A^b \bmod p$).

Alternatively, Eve may try to solve the **Computational Diffie-Hellman (CDH) problem**: given $g^a \bmod p$ and $g^b \bmod p$, compute $g^{ab} \bmod p$ directly without finding a or b — but no efficient algorithm is known for this either.

Computational Difficulty:

Algorithm	Time Complexity
Brute force (trial $a = 1, 2, ...$)	$O(p)$ — exponential in bit-length of p
Baby-step giant-step	$O(\sqrt{p})$ — still exponential in half the bit-length
Pohlig-Hellman	Efficient only when $p - 1$ has small prime factors
Index Calculus / GNFS	Sub-exponential: $\exp(O((\log p)^{1/3}(\log\log p)^{2/3}))$

For the toy example ($p = 23$), Eve can brute-force trivially. However, for real DH deployments with $p \approx 2^{2048}$, even the best sub-exponential algorithms require computational effort far beyond practical capability. **This computational hardness of the DLP is the security foundation of Diffie-Hellman.**

(c) Consistent Hashing on Virtual Ring, M=13 (10 marks)

Ring: positions $0, 1, \ldots, 12$.

Hash functions for servers: $f(\text{ip}) = \text{ip} \bmod 13$, $g(\text{ip}) = ((\text{ip} + 51) \times 150) \bmod 13$

Server	ip	f(ip)	g(ip)	Ring positions
s1	2	2	$(53\times150) \bmod 13 =$ $(1 \times 7) \bmod 13 = 7$	**2, 7**
s2	30	4	$(81\times150) \bmod 13 =$ $(3 \times 7) \bmod 13 = 8$ (since 81 mod 13 = 3, 150 mod 13 = 7) → 21 mod 13 = 8	**4, 8**

Server	ip	f(ip)	g(ip)	Ring positions
s3	80	2	$(131 \times 150) \bmod 13 = (1 \times 7) \bmod 13 = 7$ (since $131 \bmod 13 = 1$)	**2, 7**

Note: s3 hashes to the same positions (2, 7) as s1 — a collision. Both servers co-exist at those positions.

Hash functions for data: $f(d) = d \bmod 13$, $g(d) = (d + 51) \bmod 13$

Data items are placed on the **nearest server clockwise** from each hash position.

Trace of events (ring state and data assignments):

After s1 started: ring = {s1@2, s1@7}

After s2 started: ring = {s1@2, s2@4, s1@7, s2@8}

d=50 added $f(50) = 11$, $g(50) = 101 \bmod 13 = 10$

- pos 11 → clockwise → pos 12 → 0 → 1 → 2 = **s1**
- pos 10 → clockwise → 11 → 12 → 0 → 1 → 2 = **s1**

d=50 → s1 (both replicas)

d=70 added $f(70) = 5$, $g(70) = 121 \bmod 13 = 4$

- pos 5 → clockwise → 6 → 7 = **s1**
- pos 4 → exactly **s2**

d=70 → replica 1: s1, replica 2: s2

d=90 added $f(90) = 12$, $g(90) = 141 \bmod 13 = 11$

- pos 12 → 0 → 1 → 2 = **s1**
- pos 11 → 12 → 0 → 1 → 2 = **s1**

d=90 → s1 (both replicas)

After s3 started: ring = {s1/s3@2, s2@4, s1/s3@7, s2@8}

d=30 added $f(30) = 4$, $g(30) = (81) \bmod 13 = 3$

- pos 4 → exactly **s2**
- pos 3 → 4 = **s2**

d=30 → s2 (both replicas)

d=40 added $f(40) = 1$, $g(40) = 91 \bmod 13 = 0$

- pos 1 → 2 = **s1/s3**
- pos 0 → 1 → 2 = **s1/s3**

d=40 → s1 or s3 at position 2 (both replicas)

After s3 fails: ring = {s1@2, s2@4, s1@7, s2@8} (Data that was solely on s3 must be recovered from replicas. Since s3 shared positions with s1, s1 retains all data previously at positions 2 and 7.)

d=10 added $f(10) = 10$, $g(10) = 61 \bmod 13 = 9$

- pos $10 \to 11 \to 12 \to 0 \to 1 \to 2 =$ **s1**
- pos $9 \to 10 \to 11 \to 12 \to 0 \to 1 \to 2 =$ **s1**

d=10 → s1 (both replicas)

d=60 added $f(60) = 8$, $g(60) = 111 \bmod 13 = 7$

- pos $8 \to$ exactly **s2**
- pos $7 \to$ exactly **s1**

d=60 → replica 1: s2, replica 2: s1

Final Server Assignments Summary

Data item	Replica 1	Replica 2
d=50	s1	s1
d=70	s1	s2
d=90	s1	s1
d=30	s2	s2
d=40	s1	s1
d=10	s1	s1
d=60	s2	s1

Is Load Balanced? No, the load is not balanced. With only s1 and s2 remaining (s3 failed), and s1 occupying positions 2 and 7 while s2 occupies positions 4 and 8:

- s1 serves positions $[9, 2]$ (clockwise arc) — a span of 7 positions
- s2 serves positions $[3, 8]$ (clockwise arc) — a span of 6 positions

s1 holds the primary replica of d=50, d=70, d=90, d=40, d=10, d=60 (6 primary assignments) while s2 holds d=30 as both replicas and one replica each of d=70 and d=60. **s1 is significantly more loaded than s2.** The collision of s3 with s1 (same ring positions) contributed to this imbalance — in practice, virtual nodes (vnodes) with more diverse hash positions would provide better distribution.

End of 2024 Model Answers